

Creating a CNN on MATLAB for rice image classification

ABSTRACT

A CNN was created to help classify images of rice grains according to their variety. During the process, we tested with different pre trained models available on MATLAB, performed dropout analysis and drew results and conclusions from them. Ultimately, it was found that with the help of fine tuning, all pre trained models were able achieve 100% accuracy.

PERFORMANCE METRICS

The 3 major aspects we focused on while training the CNN were accuracy, memory, and time taken. Accuracy was divided into overall validation accuracy, macro accuracy, macro F1 score, macro precision, and macro recall. Memory was further divided into memory used during draining and before training. Time taken was divided into total training time and average inference per image.

RESULTS TABLE

Below the result table which we acquired after testing all 5 different models with different learning parameters. Once we identified the best training parameters, we coalesce them and trained the model with them. the table below shows us the results we obtained on doing so

EXPERIMENTAL

METHODOLOGY ABSTRACT

Instead of building a model from scratch we will be building a secondary model on top of a primary pre trained model available on MATLAB. We will compare 5 different pre trained models - AlexNet, GoogLeNet - ImageNet, GoogLeNet - places 365, Vgg 19, DenseNet 201, and NasNet Large. Each model was tested with different learning parameters to identify the best synergy of parameters for each model separately.

FUTURE WORK

Time is the crux while working with CNNs. Further development is required in the architecture of the pre trained models which specialize in speed, resource intensiveness, accuracy, and overall efficiency. A balance of the trio will be desirable but is challenging because there are tradeoffs while trying to increase any one of the aspect (memory, time, and accuracy). A model can be created from scratch which specializes in identifying agricultural objects to further streamline the pre trained model which will lead to a more confident accuracy percentage since we have a high deviation rate right now because our pre trained is very diverse in nature regarding object recognition. If we ever complicate our existing problem, we can investigate complicated models like NasNet Large etc. However, right now experimenting with complicated models is a waste of resources since we can achieve 100% generations which require comparatively lower computational power and time. Given enough time, we could run the other models except AlexNet can be run with 75,000 images and see the result.

PROBLEM AND DATA

We have a total of 75,000 of rice images which we wish to classify into 5 varieties given before hand to us. The varieties of rice images, their format, size, and resolution are as follows:



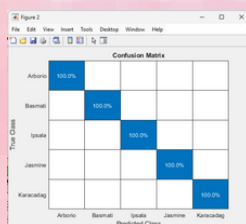
Arborio - JPG - 3.5 Kbs - 250 * 250
Basmati - JPG - 3.5 Kbs - 250 * 250
Ipsala - JPG - 4.5 Kbs - 250 * 250
Jasmine - JPG - 3.5 Kbs - 250 * 250
Karacadag - JPG - 3.4 Kbs - 250 * 250

RESULTS

For large datasets, training after a single epoch becomes redundant since convergence is achieved well in advance. MiniBatch size is directly proportional to memory required 100% accuracy was achieved in all models except NasNet Large The accuracy of the models has a deviation of 0.5% - 1.2% on an average with each training session which makes our experiment biased in nature Except DenseNet 201, all remaining models have smooth training graphs. This indicates that the model is still making an effort to make itself more robust by trying to identify newer patterns even in the latter stages of the training. During the dataset comparison, we know that the architecture of the GoogLeNet models were same. This aims that the issue lies with the pre training done signifying that ImageNet is a more effective dataset than places 365 in our problem scenario. Overall good convergence was achieved across all models with multiple 100% performers The loss function graph is a poor performance metric. In a nutshell, it indicates how well the model is learning. In some cases, the model might have very low accuracy, but it might display a 0% loss in the graph indicating that model is learning wrong patterns over time and becoming confident with its erroneous predictions. If looked at the graph alone, it might be misleading while assessing the performance of the model.

CONCLUSIONS

Despite having one of the best resources in terms of GPUs, CPUs, and PCs (hardware); the process still proves to be very taxing in terms of time while testing with complex pre trained models and very large datasets. The problem in our case was simple and straightforward in nature and therefore passing it through a heavy pre trained model was merely nothing but overengineering since 100% was achieved with AlexNet (the simplest of the models) While accuracy and memory can still be tackled with fine tuning and informed resource employment, time is the most crucial and most difficult aspect out of the three to deal with Performance metrics need to be selected carefully to avoid leading to faulty analysis as seen with loss function graph plotted ImageNet proves to be a better starting point than places 365 in our case since ImageNet specializes more in object recognition while places 365 is focused more on scene recognition making ImageNet a better fit because we are dealing with object recognition (rice images). This was also confirmed by the results we obtained in the dataset comparison done on GoogLeNet model.



Performance metrics - AlexNet trained with 75,000 images									
Learning Parameters	Accuracy	Precision	Recall	F1 Score	Training time (min)	Inference time per image (sec)	Iterations per epoch/epoch		
Optimizer									
adam	0.9949	0.995	0.9949	0.9949	14.26	0.0007	52		
sgdm	0.9838	0.984	0.9838	0.9838	14.1	0.001	52		
rmsprop	0.9863	0.9865	0.9863	0.9863	15.97	0.0013	52		
InitialLearningRate									
0.1	0.2	NaN	0.2	NaN	10.7	0.0011	52		
0.01	0.2	NaN	0.2	NaN	14.89	0.0009	52		
0.001	0.2016	NaN	0.2016	NaN	14.61	0.0012	52		
0.0001	0.9952	0.9953	0.9952	0.9952	11.9	0.0014	52		
0.00001	0.9888	0.989	0.9888	0.9888	13.71	0.0009	52		
0.000001	0.961	0.9621	0.961	0.9597	9.36	0.0006	52		
LearnRateSchedule									
none	0.996	0.996	0.996	0.996	13.81	0.001	52		
piecewise	0.9959	0.9959	0.9959	0.9959	9.2	0.0007	52		
Training validation split									
50/50	0.9952	0.9952	0.9952	0.9952	13.76	0.0006	52		
60/40	0.9963	0.9963	0.9963	0.9963	14.18	0.0012	52		
70/30	0.9961	0.9962	0.9961	0.9961	9.961	0.0006	52		
80/20	0.9952	0.9953	0.9952	0.9952	15.58	0.001	52		

Model Name	Performance metrics - Best recorded data of all pre trained models									
	Overall validation accuracy	Macro accuracy	Macro Precision	Macro Recall	Macro F1 Score	Training time (sec)	Inference time per image (sec)	Initial total memory used (MB)	Total memory used for training (MB)	Epochs/iteration per epoch
AlexNet	100.00%	100.00%	100.00%	100.00%	100.00%	581.4	0.0035	18515.19	-376.48	10 epochs / 29
GoogLeNet - ImageNet	100.00%	100.00%	100.00%	100.00%	100.00%	66.78	0.003	12883.03	1792.89	10 epochs / 14
GoogLeNet - places 365	100.00%	100.00%	100.00%	100.00%	100.00%	45.75	0.0067	11817.86	3712.27	10 epochs / 14
Vgg 19	100.00%	100.00%	100.00%	100.00%	100.00%	201.77	0.0082	22262.75	341.61	10 epochs / 14
DenseNet 201	100.00%	100.00%	100.00%	100.00%	100.00%	344.06	0.0211	18635.03	-2063.13	10 epochs / 14